

Script-Controlled ACL - Restrict Record Access Based on Field Value

Project Documentation

Team Members	Shivani P; Sheerin Farhath; Seshriya; Shalini; Shanthini
Platform	ServiceNow Administrator
Core Module	Institution Details ('u_institution_details')
Security Mechanism	Access Control Lists (ACLs) with conditions and scripted logic
Implementation Evidence	72-page ServiceNow screenshot evidence supplied by the team

Project purpose: restrict record access according to a record field value while allowing designated administrative or authorized-role access. The submitted implementation uses the Institution Details table, Branch-based conditions, read/write ACLs, and an advanced scripted condition to control access.

Documentation basis: The structure is adapted from the supplied Sample Project Documentation, while the implementation details and figures are based on the team's ServiceNow screenshot evidence.

1. Project Description

The project, Script-Controlled ACL - Restrict Record Access Based on Field Value, demonstrates how ServiceNow Access Control Lists can be used to restrict access to records based on field-level or record-level conditions. The project creates an Institution Details table and protects its records using ACL rules. The submitted evidence shows the complete flow from table creation through ACL configuration and security-rule verification.

Problem Statement

In a shared ServiceNow table, different users may need different levels of access to records. A simple role-only permission can be too broad when access must also depend on a particular field value. The project addresses this requirement by combining ServiceNow ACL decision logic with a Branch condition and an advanced script so that access can be granted or denied according to the configured security policy.

Objectives

- Create a custom Institution Details table in ServiceNow.
- Configure required fields including Branch and user/reference-based fields.
- Provide Branch values for the records, including EEE, ECE and CSE.
- Create read and write ACL rules for the custom table.
- Use a Branch = EEE condition together with scripted access logic.
- Verify read and write security decisions using ServiceNow security-rule verification.

Project Scenario

For the demonstrated scenario, access to EEE records is restricted so that administrators retain access while the configured authorized role can access the relevant records. Other users are denied according to the script. A separate write ACL is configured with its own required role and verification.

2. Technical Architecture

The solution follows a ServiceNow platform-centric architecture. Users interact with the ServiceNow interface; the request reaches the platform's security evaluation layer; ACL rules then evaluate role information, configured record/field conditions, and scripted logic before the platform returns an allow/deny decision.

USER / ADMIN	SERVICE NOW PLATFORM	ACCESS DECISION
User role / session	Institution Details table `u_institution_details`	ALLOW / DENY
Requested operation READ or WRITE	ACL evaluation Role + Condition + Script	Record access according to policy

Main Components

- Institution Details table: custom table named `u\_institution\_details`.
- Fields: Student Roll Number, Student Name, Faculty Name, Branch, Email, Phone Number, Description, plus standard system fields.
- Choice data: Branch options include EEE, ECE and CSE.
- ACLs: read and write rules configured for the custom table.
- Conditions: field-based logic such as Branch is EEE.
- Advanced script: programmatic allow/deny decision based on the current user's role.
- Verification: ServiceNow Verify Security Rules screens show the resulting security decision.

The screenshot shows the ServiceNow 'Table - New Record' configuration page. The 'Label' is 'Institution Details' and the 'Name' is 'u_institution_details'. The 'Application' is set to 'Global'. The 'Create module' and 'Create mobile module' checkboxes are checked. The 'Add module to menu' dropdown is set to '-- Create new --', and the 'New menu name' is 'Institution Details'. The 'Remote Table' checkbox is unchecked. Below the configuration fields, there is a 'Columns' tab with a 'Table Columns' section. The 'Table Columns' section has a search bar and a table with columns: Column label, Type, Reference, Max length, Default value, and Display. The table is currently empty, with a row for 'Insert a new row...'. The 'Submit' and 'Cancel' buttons are at the bottom.

Figure 1. Custom Institution Details table configuration captured in ServiceNow.

3. Project Workflow

Phase	Activity	Output
1. Requirement	Identify need for field-value-based record restriction.	Security requirement
2. Data Model	Create 'u_institution_details' and define fields.	Custom table + fields
3. Choice Data	Configure Branch values and create test records.	EEE / ECE / CSE records
4. Roles	Create/assign project-specific user roles for ACL evaluation.	Role-based access context
5. Read ACL	Configure read ACL with required role, Branch condition and script.	Controlled read access
6. Write ACL	Configure write ACL with required role and security conditions.	Controlled write access
7. Verification	Use Verify Security Rules for read and write operations.	Allow/deny verification evidence
8. Testing	Check configured behavior and capture screenshots.	Implementation evidence

4. Milestone 1 - Data Model and Table Creation

The first implementation milestone establishes the custom data structure. The evidence shows a new ServiceNow table named Institution Details with the internal name `u\_institution\_details`. The table is used as the protected resource for the subsequent ACL configuration.

ServiceNow recommends creating custom tables in scoped applications. To learn more about creating scoped applications, click [here](#).

A table is a collection of records in the database. Each record corresponds to a row in a table, and each field on a record corresponds to a column on that table. Applications use tables and records to manage data and processes. [More info](#)

* Label: Institution Details
* Name: u_institution_details
Extends table:

Application: Global
Create module: ☒
Create mobile module: ☒
Add module to menu: -- Create new --
New menu name: Institution Details
Remote Table: ☐

Columns | Controls | Application Access

Table Columns for text Search

Dictionary Entries

Column label	Type	Reference	Max length	Default value	Display
Insert a new row...					

Submit Cancel

Figure 2. Creation of the Institution Details table in ServiceNow.

ServiceNow

Table - Institution Details

Additional actions

Columns | Controls | Application Access

Table Columns for text Search

Dictionary Entries

Column label	Type	Reference	Max length	Default value	Display
Updated	Date/Time	(empty)	40	false	false
Created by	String	(empty)	40	false	false
Created	Date/Time	(empty)	40	false	false
Sys ID	Sys ID (GUID)	(empty)	32	false	false
Updated by	String	(empty)	40	false	false
Updates	Integer	(empty)	40	false	false
Student Roll Number	String				false
Student Name	Reference	User			false
Faculty Name	Reference	User			false
Branch	Choice				false
Email	String				false
Phone Number	String				false
Description	String				false
Insert a new row...					

Delete Update Delete All Records

Figure 3. Institution Details fields configured in the custom table.

5. Milestone 2 - Choice Values and Sample Records

The project then configures the Branch field as a choice field. The submitted evidence shows selectable values including EEE, ECE and CSE. Sample Institution Details records are created so that the ACL behavior can be demonstrated against actual data.

The screenshot shows the 'Institution Details - Create Created' form in ServiceNow. The form has the following fields and values:

- Student Roll Number: STU200
- Faculty Name: Allyson Gillisple
- Branch: EEE (selected from a dropdown menu)
- Phone Number: (empty)
- Student Name: Alejandra Prenatt
- Email: (empty)
- Description: (empty)

A 'Submit' button is located at the bottom left of the form.

Figure 4. Institution Details record creation with Branch set to EEE.

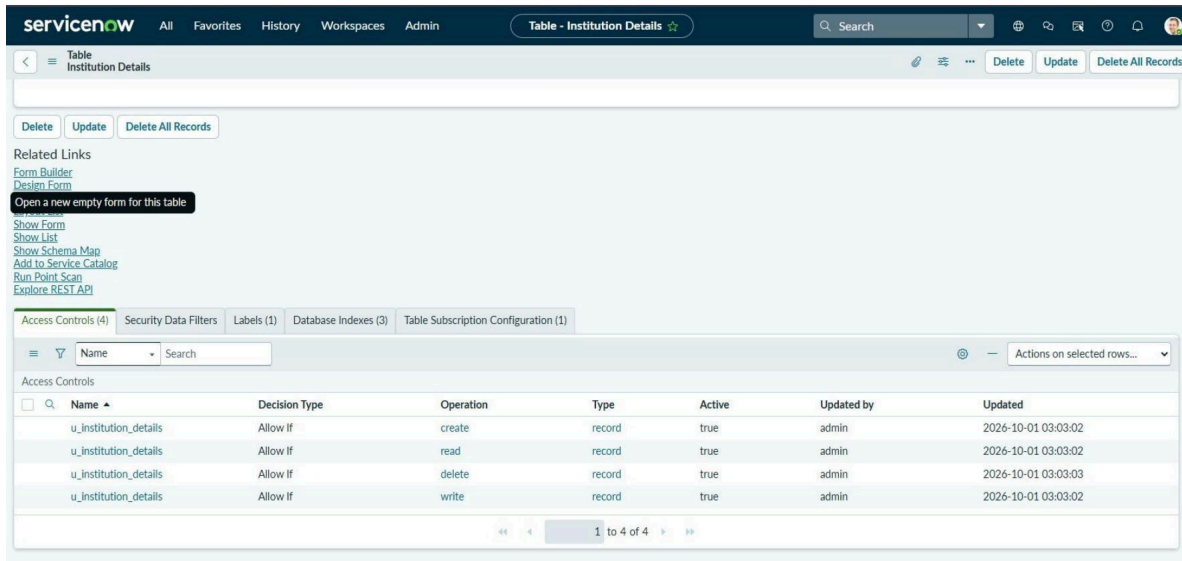
The screenshot shows the 'Institution Details' list view in ServiceNow. The table has the following columns: Branch, Description, Email, Faculty Name, Phone Number, Student Name, and Student Roll Number. The data is as follows:

Branch	Description	Email	Faculty Name	Phone Number	Student Name	Student Roll Number
EEE			Allyson Gillisple		Alejandra Prenatt	STU200
ECE			Abraham Lincoln		Abel Tuter	STU100

Figure 5. Institution Details list showing stored records and their Branch values.

6. Milestone 3 - Roles and Access Control

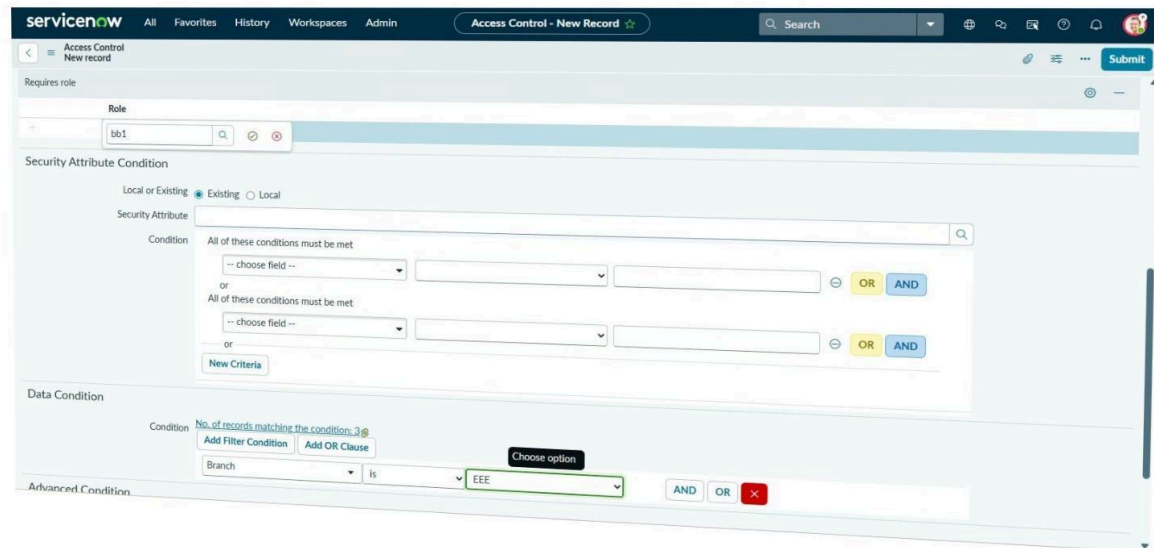
The implementation creates and manages users and roles so ACL rules can evaluate different access contexts. The evidence then moves to the Access Control list, where rules for the custom `u\_institution\_details` table are configured. The ACL form includes the operation, active state, decision type, required role, conditions, and an Advanced Condition script area.



The screenshot shows the ServiceNow interface for the 'Table - Institution Details'. The 'Access Controls' tab is active, displaying a list of four rules for the 'u_institution_details' table. Each rule has a checkbox, a search icon, a name, a decision type, an operation, a type, an active status, an updated by user, and an updated timestamp.

☐	Search	Name	Decision Type	Operation	Type	Active	Updated by	Updated
☐		u_institution_details	Allow If	create	record	true	admin	2026-10-01 03:03:02
☐		u_institution_details	Allow If	read	record	true	admin	2026-10-01 03:03:02
☐		u_institution_details	Allow If	delete	record	true	admin	2026-10-01 03:03:03
☐		u_institution_details	Allow If	write	record	true	admin	2026-10-01 03:03:02

Figure 6. Access Control rules list in ServiceNow.



The screenshot shows the 'Access Control - New Record' form in ServiceNow. The form includes sections for 'Requires role' (with a role dropdown set to 'bb1'), 'Security Attribute Condition' (with 'Local or Existing' set to 'Existing'), and 'Data Condition' (with a condition set to 'No. of records matching the condition: 3'). The 'Advanced Condition' section is also visible at the bottom.

Figure 7. New Access Control rule form for the custom table.

7. Milestone 4 - Field-Value Condition and Script

The key implementation step is the combination of a field-level data condition and a scripted ACL decision. The submitted evidence shows Branch is EEE as the data condition. The Advanced Condition contains an ECMAScript function that explicitly allows administrators, allows the configured 'bb1' role, and denies other users.

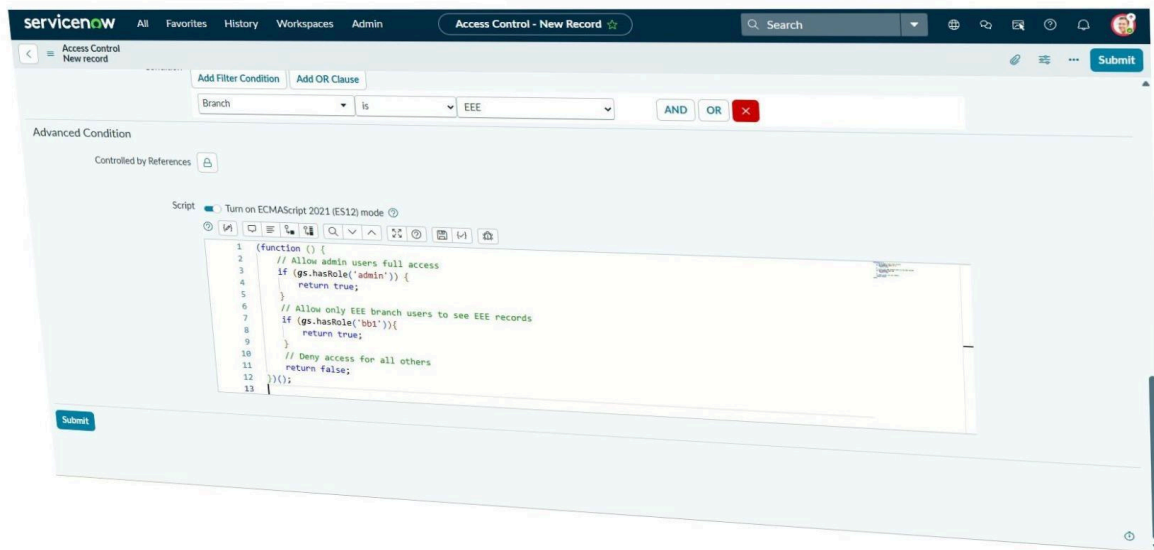


Figure 8. Branch = EEE condition and the advanced script used by the ACL.

Script logic represented in the submitted evidence:

```
(function () {  
  // Allow admin users full access  
  if (gs.hasRole('admin')) {  
    return true;  
  }  
  
  // Allow the configured role to access the tested records  
  if (gs.hasRole('bb1')) {  
    return true;  
  }  
  
  // Deny access for all others  
  return false;  
})();
```

8. Milestone 5 - Read ACL Verification

After configuring the read ACL, ServiceNow's security-rule verification is used to inspect the resulting decision. The submitted evidence shows a Read (Allow) result for `u\_institution\_details`, providing direct evidence that the configured rule is being evaluated in the demonstration context.

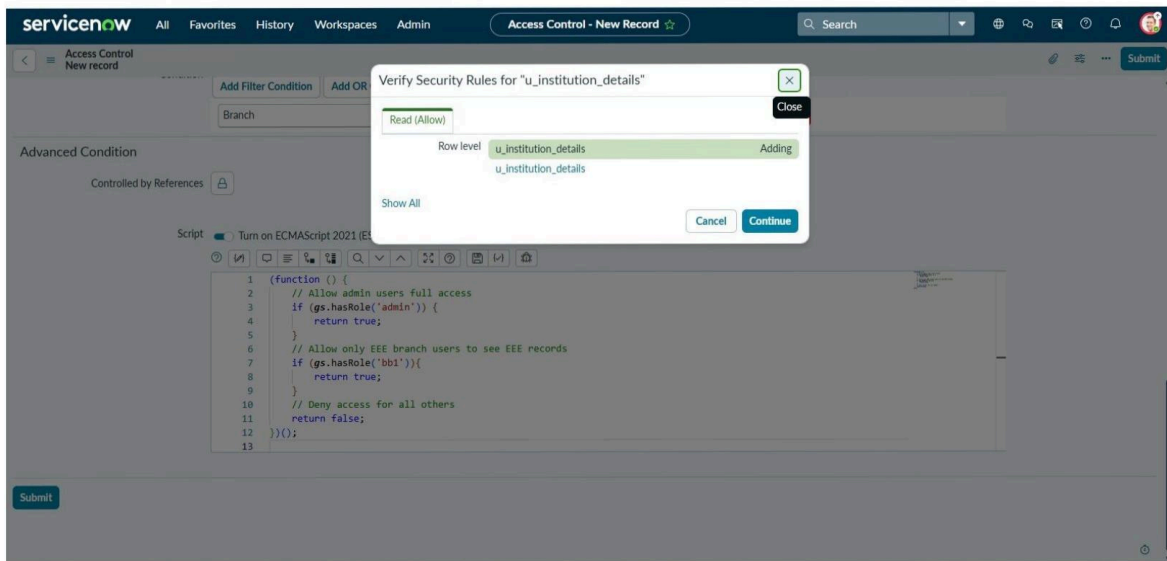


Figure 9. Scripted read ACL and its configuration.

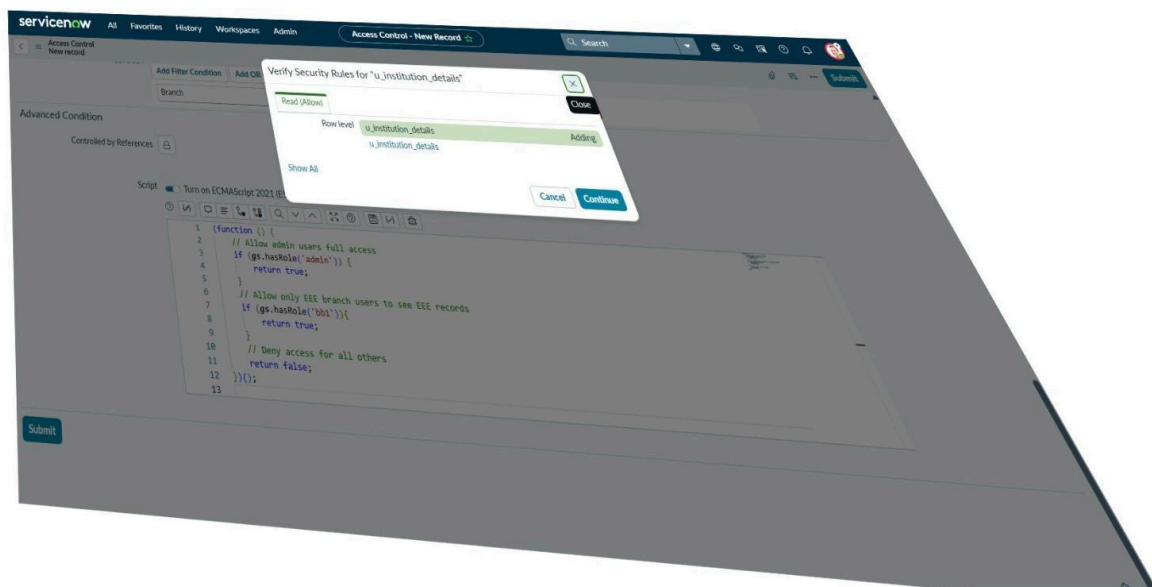


Figure 10. Verify Security Rules dialog showing Read (Allow).

9. Milestone 6 - Write ACL Configuration

A separate write ACL is configured to control modification access. The submitted evidence shows the Write operation, the `u\_institution\_details` resource, and a required role in the ACL conditions. This demonstrates that the project does not rely on a single permission for all operations; read and write access are controlled independently.

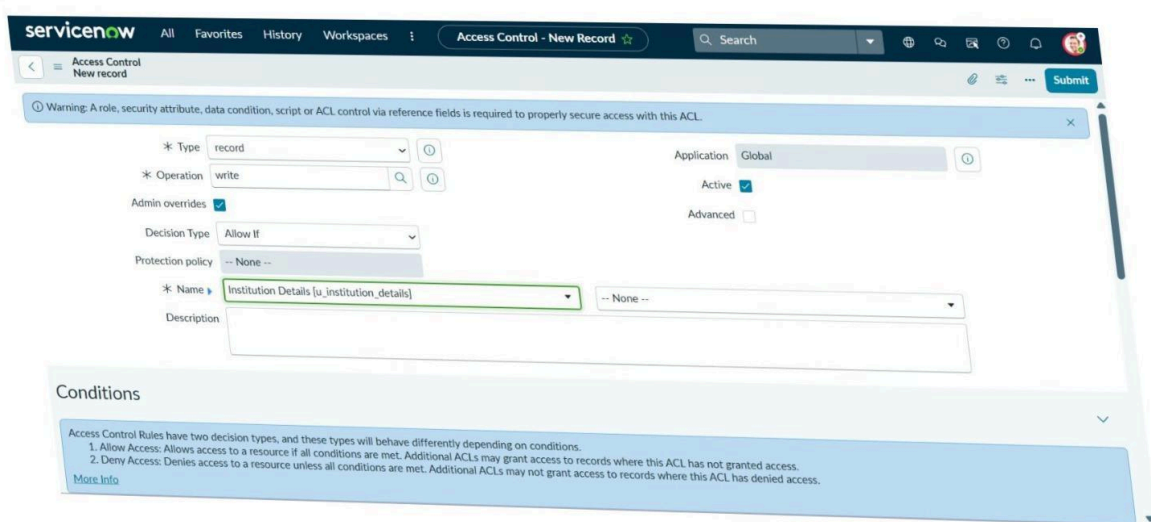


Figure 11. Write Access Control rule configuration.

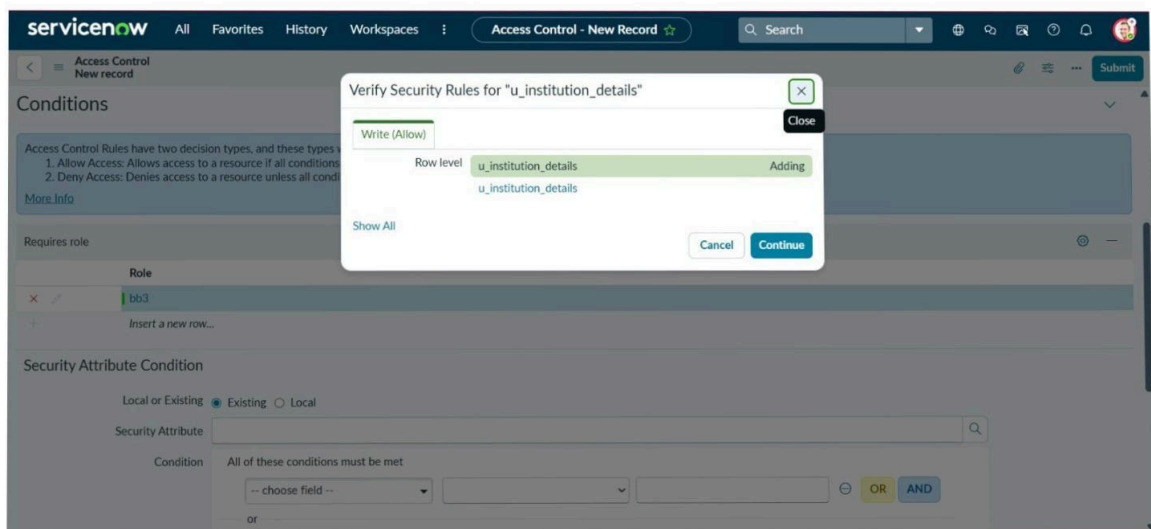


Figure 12. Verify Security Rules dialog showing Write (Allow) for the custom table.

10. Testing Strategy

Testing for this project is security/functional testing rather than conventional application load testing. The objective is to confirm that the ACL rules evaluate correctly for the intended operation, role, and record condition. Each test uses the ServiceNow interface and the built-in security-rule verification evidence.

ID	Test Area	Procedure	Expected Result
TC-01	Table configuration	Open 'u_institution_details' and confirm the custom fields exist.	Configuration is present.
TC-02	Branch choices	Inspect Branch choice list.	EEE, ECE and CSE choices are available.
TC-03	Read ACL setup	Inspect read ACL role, condition and script.	Expected read policy is configured.
TC-04	Read verification	Run Verify Security Rules for the read rule.	Read (Allow) is shown for the configured context.
TC-05	Write ACL setup	Inspect write operation and required role.	Expected write policy is configured.
TC-06	Write verification	Run Verify Security Rules for the write rule.	Write (Allow) is shown for the configured context.

Performance testing note: The team's supplied evidence does not include JMeter, Locust, k6, CPU/memory monitoring, or measured throughput. Therefore, numerical performance claims are not included in this project documentation.

11. Implementation Evidence Summary

Evidence	What it proves
Table creation	Custom `u_institution_details` table exists.
Field configuration	Institution Details fields are configured.
Branch choices	Branch values are configured for the test data.
Record creation	Test records are created in the table.
ACL configuration	Access Control rules exist for the custom table.
Branch = EEE condition	Record-level/field-value restriction is configured.
Advanced script	Role-aware allow/deny logic is configured.
Read verification	ServiceNow reports Read (Allow) for the tested rule.
Write verification	ServiceNow reports Write (Allow) for the tested rule.

Together, the screenshots show the complete implementation chain: data model -> test records -> roles -> ACL configuration -> Branch-based restriction -> scripted decision -> security-rule verification.

12. Limitations and Security Considerations

- The solution is tied to the roles and security configuration present in the ServiceNow instance.
- The document intentionally does not publish instance URLs or login credentials.
- ACL behavior should be re-verified whenever roles, table fields, or security rules are changed.
- The current evidence demonstrates a Branch = EEE scenario; additional field values can be supported by extending the ACL policy.
- No external load/performance testing was conducted in the supplied evidence.

Future Enhancements

- Add additional branch-specific ACL policies for other organizational groups.
- Introduce a more data-driven access mapping between user role and permitted Branch.
- Add a dedicated test matrix covering positive and negative role combinations.
- Capture formal performance measurements if a performance-testing requirement is introduced.
- Package the configuration as a managed ServiceNow application/update set for repeatable deployment.

Expected Benefits

- Reduces unauthorized access to sensitive records.
- Provides field-aware access control instead of relying only on broad roles.
- Makes the security policy visible and verifiable through ServiceNow ACL tooling.
- Separates read and write permissions so each operation can be controlled independently.

13. Conclusion

The project demonstrates a ServiceNow security design in which record access is restricted according to field value and user role. The custom `Institution Details` table acts as the protected resource, while ACL rules provide operation-specific control. The read policy combines the Branch = EEE condition with an advanced script that grants access to administrators and the configured `bb1` role while denying other users. A separate write ACL provides independent control over modification access.

The submitted screenshots provide implementation evidence for table creation, field configuration, record creation, ACL configuration, scripted logic, and both read and write security-rule verification. This makes the project suitable for a demonstration-focused ServiceNow security submission.

Team

Shivani P | Sheerin Farhath | Seshriya | Shalini | Shanthini

Final implementation identifier: `u\_institution\_details`

Primary security scenario: Branch = EEE with scripted role-based access decision

Verification evidence: ServiceNow Verify Security Rules - Read (Allow) and Write (Allow)